

RUBINA DANGOL MAHARJAN AI FELLOWSHIP FUSEMACHINE



10th May 2026

Week-2 Task-3

Mastering Factor VIII: Concurrency (Twelve-Factor App)

Presented to

Prabhat Ale
FuseMachine

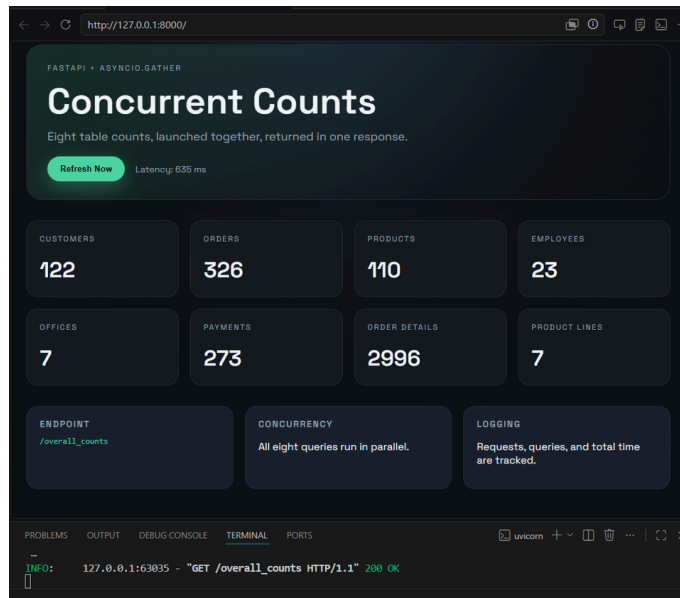
Presented by

Rubina Maharjan

TABLE OF CONTENTS

About the assignment
Individual API Endpoints
CRUD Layer Design
Aggregated Endpoint
Use of Concurrency
Logging Implementation
Dashboard Page
Error Handling and Robustness
Testing
Error journal
Learning Outcome
Conclusion

FastAPI dashboard that fetches table row counts concurrently using `asyncio.gather()`. It improves speed with modular routes, database handling, and logging.

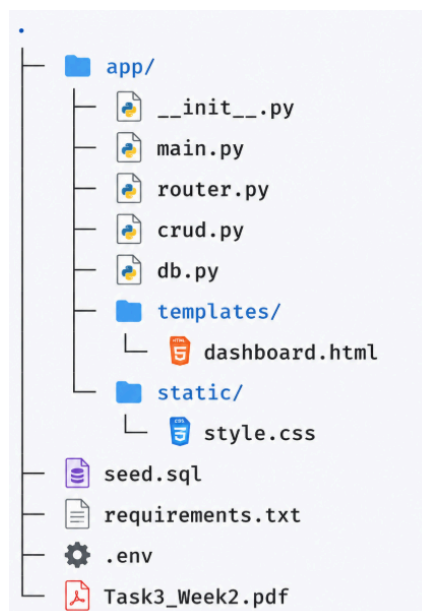


Database name: mydb

Goal: The system should be able to:

- To create eight individual API endpoints for eight database tables.
- To keep the project modular using separate router and CRUD layers.
- To create one aggregated endpoint named `/overall_counts`.
- To use `asyncio.gather()` so that all database count queries run concurrently.
- To avoid slow sequential database fetching.
- To implement logging at both endpoint and CRUD levels.
- To return a combined JSON response containing all table counts.
- To optionally display the aggregated counts in a simple HTML dashboard.

Files:



Individual API Endpoints

Endpoints for Customers, Orders, Products, Employees, Offices, Payments, Order Details, and Product Lines.

Table Name	Endpoint	Purpose
Customers	GET /customers/count	Returns total customers
Orders	GET /orders/count	Returns total orders
Products	GET /products/count	Returns total products
Employees	GET /employees/count	Returns total employees
Offices	GET /offices/count	Returns total offices
Payments	GET /payments/count	Returns total payments
Order Details	GET /orderdetails/count	Returns total order details
Product Lines	GET /productlines/count	Returns total product lines

Each endpoint calls its own CRUD function. For example, /customers/count calls a function such as get_customers_count(), while /orders/count calls get_orders_count().

This design follows the principle of **modularity** because every endpoint is independent and reusable.

CRUD Layer Design

Responsible for directly communicating with the database

Instead of writing database queries inside the route handlers, the queries are separated into individual functions inside crud.py.

Example CRUD functions include:

```
async def get_customers_count() -> int:
    return await _count_table("customers")

async def get_orders_count() -> int:
    return await _count_table("orders")

async def get_products_count() -> int:
    return await _count_table("products")

async def get_employees_count() -> int:
    return await _count_table("employees")

async def get_offices_count() -> int:
    return await _count_table("offices")

async def get_payments_count() -> int:
    return await _count_table("payments")

async def get_orderdetails_count() -> int:
    return await _count_table("orderdetails")

async def get_productlines_count() -> int:
    return await _count_table("productlines")
```

This separation improves code readability and maintainability. If a query needs to be changed in the future, only the CRUD function has to be updated instead of changing the router logic.

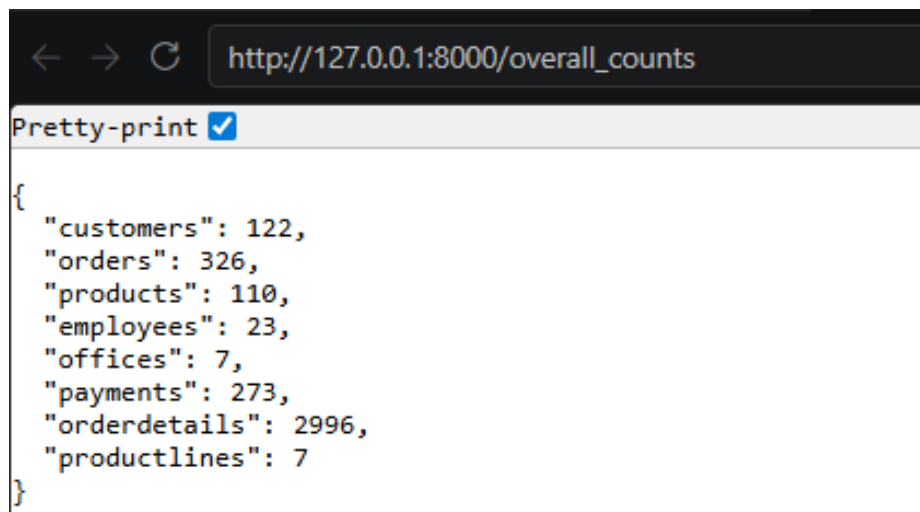
Aggregated Endpoint

overall_counts - create an aggregated endpoint

```
@router.get("/overall_counts")
async def overall_counts(request: Request) -> Dict[str, int]:
    logger.info("request start", extra={"path": request.url.path})
    start = time.perf_counter()
```

This endpoint works as the main dashboard endpoint. It returns the counts from all eight database tables in a single JSON response

Response format is:



```
{
  "customers": 122,
  "orders": 326,
  "products": 110,
  "employees": 23,
  "offices": 7,
  "payments": 273,
  "orderdetails": 2996,
  "productlines": 7
}
```

This endpoint should not fetch the data sequentially. Instead, all eight count queries should be started together using `asyncio.gather()`

Use of Concurrency

NOT fetch data sequentially.

In a sequential approach, the API would wait for one database query to finish before starting the next query. This creates unnecessary delay.

Example:

A wrong sequential approach would work like this

```
Query customers → wait  
Query orders → wait  
Query products → wait  
Query employees → wait
```

This is inefficient because the server remains idle while waiting for each database response.

The correct concurrent approach starts all eight tasks at the same time:

```
tasks = [  
    crud.get_customers_count(),  
    crud.get_orders_count(),  
    crud.get_products_count(),  
    crud.get_employees_count(),  
    crud.get_offices_count(),  
    crud.get_payments_count(),  
    crud.get_orderdetails_count(),  
    crud.get_productlines_count(),  
]
```

Using `asyncio.gather()` allows FastAPI to wait for all queries together instead of waiting for each query one by one. This improves response speed and makes the API more scalable.

Logging Implementation

Logging in the endpoint layer, CRUD layer, and concurrency endpoint.

Logging was implemented to track:

- Incoming API requests.
- Successful responses.
- Database query start and completion.
- Database errors.
- When concurrent tasks start.
- When `asyncio.gather()` completes.
- Total response time for `/overall_counts`.

Example:

```
@router.get("/customers/count")
async def customers_count(request: Request) -> Dict[str, int]:
    logger.info("request start", extra={"path": request.url.path})
```

Logging is useful because it helps developers monitor application behavior, debug issues, and measure performance.

Dashboard Page

A simple HTML dashboard page served by FastAPI

The dashboard fetches data from the `/overall_counts` endpoint and displays the counts in cards. The dashboard improves usability because users do not need to manually call the API using Postman or a browser URL. Instead, they can open:

```
@app.get("/", response_class=HTMLResponse)
async def dashboard(request: Request) -> HTMLResponse:
    logger.info("dashboard view", extra={"path": request.url.path})
    return templates.TemplateResponse("dashboard.html", {"request": request})
```

The dashboard then displays all table counts visually.

Error Handling and Robustness

The API should handle empty tables gracefully

If a table contains no records, the count query should return 0 instead of causing an error.

The application should also handle database errors properly using try-except blocks. If a query fails, the error should be logged, and the API should return a meaningful response instead of crashing.

This makes the application more robust and reliable.

Examples:

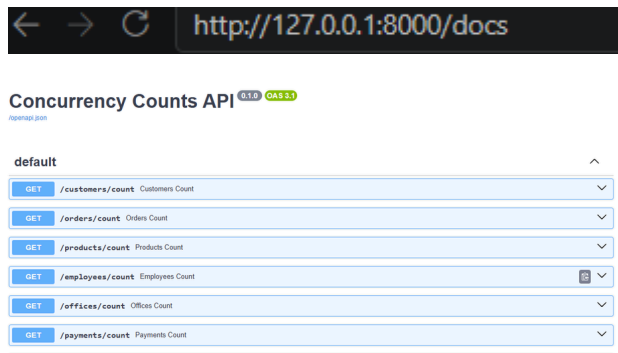
```
@router.get("/customers/count")
async def customers_count(request: Request) -> Dict[str, int]:
    logger.info("request start", extra={"path": request.url.path})
    try:
        count = await crud.get_customers_count()
        _log_success(request.url.path, 200)
        return {"customers": count}
    except Exception as exc:
        _log_failure(request.url.path, 500)
        raise HTTPException(status_code=500, detail="failed to fetch customers count") from exc
```

```
async def _count_table(table: str) -> int:
    query = f'SELECT COUNT(*) FROM "{table}"'
    logger.info("db count start", extra={"table": table})
    try:
        pool = get_pool()
        result: Any = await pool.fetchval(query)
        count = int(result or 0)
        logger.info("db count complete", extra={"table": table, "count": count})
        return count
    except Exception:
        logger.exception("db count failed", extra={"table": table})
        raise
```

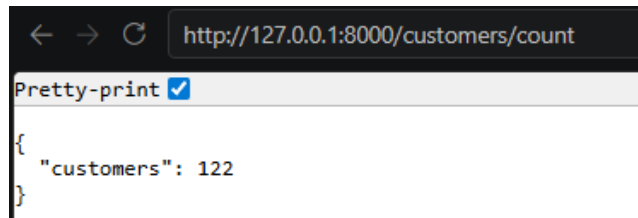
Testing

Testing API endpoints

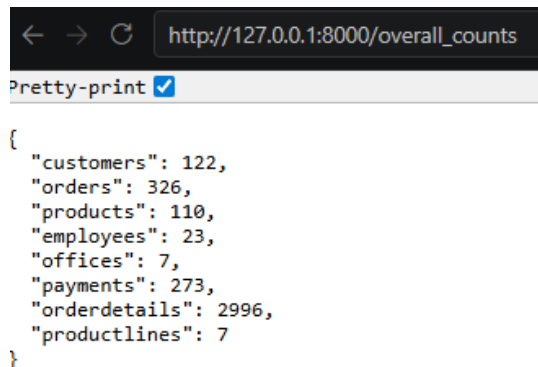
The API endpoints can be tested using FastAPI's automatic Swagger UI at:



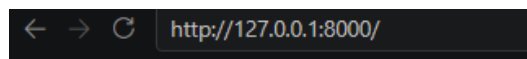
Each individual endpoint can be tested separately:



The aggregated endpoint can be tested using:



The dashboard can be tested using:



Successful testing means:

All eight individual endpoints return correct counts. /overall_counts returns all counts in one JSON response. The aggregated endpoint uses asyncio.gather(). Logs are generated for requests, database queries, and response time. Empty tables return 0.

Error journal

Error	Cause	Solution
Failed building wheel for asyncpg	The installed Python version was not fully compatible with the asyncpg build process	Using a compatible Python environment and installing an updated version of asyncpg
password authentication failed for user "user"	FastAPI app is trying to connect to PostgreSQL with the wrong database username/password.	Made FastAPI .env database credentials match the PostgreSQL credentials used by Docker

Learning Outcome

I learned how concurrency improves backend performance. I also learned how to structure a FastAPI project using modular design. Separating routes, database connection, and CRUD logic made the project cleaner and easier to understand.

The assignment also helped me understand the importance of observability through logging. By logging endpoint requests, database queries, errors, and response time, the API becomes easier to monitor and debug.

Most importantly, I learned how `async` and `await` can be used in FastAPI to prevent the server from waiting idly while database operations are running.

Conclusion

This assignment successfully demonstrates **Factor VIII: Concurrency** from the Twelve-Factor App methodology. The FastAPI application includes eight individual count endpoints and one aggregated `/overall_counts` endpoint. The aggregated endpoint uses `asyncio.gather()` to run all database count queries concurrently, improving performance compared to sequential processing.

The project also follows modular design by separating endpoint logic from CRUD/database logic. Logging was added across the application to support debugging and performance tracking. Overall, this assignment helped build a practical understanding of scalable API design, asynchronous programming, and concurrency in FastAPI.